

Question 1:

1. Explanation of fork (), wait(), and waitpid()

The fork() system call creates a new child process from the current process. After fork() is called, the parent and child continue running separately. The parent receives the child process ID as the return value, while the child receives 0. If fork() fails, it returns -1. In the manager program, fork() is used to create the employee processes.

The wait() system call allows a parent process to wait for one of its child processes to finish. It also allows the parent to collect the child's termination status so that the child does not remain as a zombie process. The manager program uses wait() to collect the results from the employee processes.

The waitpid() system call is similar to wait(), but it allows the parent to specify which child process it wants to wait for. For example, a parent can use waitpid() to wait for a specific child based on its process ID. The manager program uses wait() instead of waitpid(), because the manager can collect any employee process that finishes.

2. exec() family functions

The exec family of functions replaces the current process program with another program. In the manager program, fork() first creates a child process. The child then uses execl() to replace its program with the Employee Program. The child does not create another process when execl() is called. Instead, it becomes the employee program.

The manager uses **execl("./employee", "employee", (char*)NULL)** to run the employee executable. If execl() succeeds, the child continues as the Employee Program. If it fails, the child reports the error and terminates with a separate failure status.

3. Other POSIX APIs Used

The Employee Program uses **getpid()** to obtain the process ID of the employee. It uses **nanosleep()** to suspend the process for a randomly generated amount of time between 200 and 600 microseconds. The program also uses **exit()** to terminate the process and return its result.

The Manager Program uses **perror()** to display error messages when system calls such as **fork()**, **wait()**, or **execl()** fail. IT uses **WIFEXITED()** to determine whether an employee terminated normally and **WEXITSTATUS()** to retrieve the employee's exit status. The zombie and orphan programs also use **getpid()** and **getppid()** to obtain process IDs, and **sleep()** to keep processes running for observation.

4. Approach to Solving the Problem

First, the Manager Program starts and initializes a counter to keep track of the number of employee processes that are successfully created.

Next, the manager enters a loop that runs 100 times. During each iteration, the manager calls **fork()** to create one child process. If **fork()** is successful, the parent process continues to the

next iteration to create another employee. The child process uses **execl()** to run the Employee Program.

After **execl()** successfully runs, the child process becomes an employee and executes the Employee Program. The employee first generates a random number between 0 and 100 to represent the number of insurance policies sold. It then generates a random delay between 200 and 600 microseconds and uses **nanosleep()** to pause for that amount of time.

After the delay, the employee obtains its process ID using **getpid()** and displays its process ID and the number of policies sold. The employee then returns the sales value as its process exit status.

Once the manager has finished creating the employee processes, it waits for the successfully created employees to finish. The manager uses **wait()** to collect each employee process and its termination status.

For each collected employee, the manager uses **WIFEXITED()** to check whether the employee terminated normally. If the employee terminated normally, the manager uses **WEXITSTATUS()** to retrieve the number of policies sold. The manager adds this value to the total sales and increases the number of successfully completed employees.

Finally, after all successfully created employees have been collected, the manager calculates the average number of policies sold by dividing the total sales by the number of successfully completed employees. The manager then displays the total sales and the average.

Question 2:

1. Definition and Explanation of a Zombie Process

A zombie process is a child process that has finished executing but still has an entry in the process table because its parent has not collected its termination status. The child is no longer running, but information about the terminated process remains until the parent calls **wait()** or **waitpid()**.

2. How the Program Creates a Zombie Process

The zombie program first uses **fork()** to create a child process. The child process prints its process ID and then terminates by calling **exit(0)**. The parent process does not call **wait()** or **waitpid()**. Instead, the parent remains running in an infinite loop. Because the child has terminated but the parent has not collected its termination status, the child becomes a zombie process.

3. Commands and Procedure Used to Verify the Process is a Zombie

First, the program was compiled using the following command:

```
gcc -std=c99 zombie.c -o zombie
```

The program was then started using:

```
./zombie
```

The program displayed the parent and child process IDs. A second terminal was opened while the parent process was still running. The following command was used to check the child process:

```
ps -o pid,ppid,state,command -p 70360
```

The output showed the child process with a state of Z and the command . The Z state indicates that the process is a zombie, while indicates that the child has terminated but has not yet been collected by its parent. The parent process was still running, confirming that the child had become a zombie.

Question 3:

1. Definition and Explanation of an Orphan Process

An orphan process is a child process whose original parent process has terminated while the child is still running. When this happens, the operating system assigns the orphan process to a new parent process so that it can continue running.

2. How the Program Creates an Orphan Process

The orphan program first uses **fork()** to create a child process. The parent process prints its process ID and then terminates using **exit(0)**. The child process continues running after the parent terminates. Because the original parent has terminated while the child is still running, the child becomes an orphan process.

The child uses **getppid()** to obtain its parent process ID. After the original parent terminates, the child's parent process ID changes to the new parent assigned by the operating system.

3. Commands and Procedure Used to Verify the Process is an Orphan

First, the program was compiled using the following command:

```
gcc -std=c99 orphan.c -o orphan
```

The program was then started using:

```
./orphan
```

The program displayed the child process ID. After the original parent terminated, a second terminal was used to check the child process with:

```
ps -o pid,ppid,state,command -p
```

The child process was still running even though its original parent had terminated. The PPID of the child changed to 1, showing that the child had been adopted by the new parent process. This confirmed that the child process had become an orphan.